

Passing Is Not Shipping

Correctness, Semantic Equivalence and ReDoS Safety in
LLM-Generated Regular Expressions, and the Limits of Measuring Them

Foothills Labs
info@foothills-labs.com

August 2026

Abstract

We evaluate eleven contemporary language models on natural-language-to-regex generation over 450 tasks with three samples each (14,850 requests, \$10.95), scoring every answer on three axes: whether it satisfies the task’s tests, whether it denotes the same language as a human reference, and whether it is vulnerable to regular-expression denial of service (ReDoS).

Our headline result is a negative one about a composite metric we built from the corpus’s own metric suite and then had to abandon. A composite requiring correctness, safety and semantic equivalence appears to show that “passing is twice as easy as shipping”: models pass 38.0–47.5% of tasks but satisfy all three criteria on only 17.1–24.8%. Decomposing that gap shows that **87% of it is contributed by the semantic-equivalence term**, and a manual audit shows that term is itself dominated by reference-set error and task underspecification rather than model behaviour. Using only the two criteria that never consult a human answer key, the gap is **2.9 percentage points**: just 7.4% of functionally correct patterns are ReDoS-vulnerable.

That figure is itself informative by contrast. Joint correctness-and-security benchmarks in general code generation report far larger penalties. BaxBench finds roughly half of correct backends exploitable. SecureAgentBench reports 15.2% correct-and-secure for its best agent. The regex domain does *not* reproduce that pattern, and we report the difference rather than the similarity. Separately, every model is safer than the corpus’s own human-written reference patterns (9.0% pooled against 13.6%). We initially read that as evidence that ReDoS is endemic to human practice. Extending the identical screen to five further populations, including half a million regular expressions extracted from shipped packages, refutes it and replaces it with a sharper ordering. Controlling for task mix, patterns *written to be read* are ReDoS-prone (reusable library entries 20.1%, Stack Overflow answers 17.3%, this corpus’s reference answers 13.4%), while patterns that are *executed* in shipped packages sit at 8.9%, statistically indistinguishable from the models at 9.0%. *Vulnerability tracks whether a pattern was ever run, not whether a human or a model wrote it.*

The audit behind this is the paper’s methodological contribution. Metrics that compare generated output against a human answer key inherit that key’s error rate, a dependency documented for image and text benchmarks [Northcutt et al., 2021] but not, to our knowledge, quantified for this one. In a random sample of disagreements where a model passed every test yet was scored as semantically different, the model was clearly at fault in roughly 15% of cases. The rest split between demonstrably incorrect reference patterns (including a reference character class containing a literal `|`) and prompts that never specified the property in dispute. Because that unreliable term supplies 87% of our composite’s headline gap, the composite was measuring its own weakest component. We also show that independent binomial intervals, the default in this literature, are the wrong inferential tool when all systems answer identical items, and that a paired bootstrap resolves nine times as many pairwise comparisons on the same data.

Replicating the two reference-independent criteria on a second corpus [Ye et al., 2020] shows the qualitative finding holds and the number does not: the same eleven models leave **16.5%** of their correct patterns ReDoS-vulnerable there against 7.4% here, on tasks twenty points *easier*. The correctness-to-security penalty is a property of the corpus, not of the models.

We therefore decline to publish a ranking. We release all raw generations, scoring code and adjudications, and recommend that evaluations separate reference-dependent from reference-independent metrics, assign them different evidentiary weight, and treat any effect size as corpus-local until replicated.

Artifacts. All model outputs, scoring code, per-case adjudications and analysis scripts are available at <https://github.com/foothills-labs/regexleaderboard> [Foothills Labs, 2026b]. Every number in this paper recomputes offline from committed data with no API access.

1 Introduction

Regular expressions make an unusually good probe for language-model evaluation. The task is compact. The output runs. Correctness is partly decidable, which is rare. And a syntactically valid, test-passing answer can still be a security defect, which is what drew us to the task in the first place.

The standard protocol for this task scores a candidate against a reference by DFA equivalence [Locascio et al., 2016, Kushman and Barzilay, 2013], sometimes alongside test-based correctness. Neither criterion notices catastrophic backtracking. A pattern can satisfy every provided example, denote exactly the reference language, and still admit inputs on which matching takes time exponential in the input. That is the ReDoS vulnerability class [Davis et al., 2018], a denial-of-service vector in deployed systems and common enough in human-written code to have prompted ecosystem-scale study.

Siddiq et al. [2024a] closed that gap. Their Re(gEx|DoS)Eval introduces the corpus we use, defines the four metrics we use (PASS@ k , VULNERABLE@ k , DFA-EQ@ k and exact match), scores correctness and security jointly on them, and evaluates T5, Phi-1.5 and GPT-3.5-Turbo. The instrument in this paper is theirs. We say so plainly here because an earlier draft of this work did not, and because it determines what is left for us to contribute: not the measuring apparatus, but what happens when it is turned on the current model population, decomposed, and then turned back on itself.

1.1 Contributions

1. **A decomposition of the joint metric** (Section 4.2). The composite is the natural way to report correctness and security together and it is what the joint-benchmark literature reports (Section 2.1). We show that on this corpus the composite is dominated by its equivalence term, which supplies 87% of the headline gap, and that the term is the least trustworthy of the three. The contribution is the finding that the aggregate is uninformative here, not the aggregate.
2. **A quantified audit of reference-standard error** (Section 5.1). We sample disagreements and adjudicate them, finding that the majority of apparent semantic errors are not model errors. Northcutt et al. [2021] establish that test-set label error is pervasive and can reverse benchmark orderings; we measure it for a corpus whose labels are regular expressions, where a label error is a formal object that can be exhibited and argued with rather than a disputed annotation. Siddiq et al. [2024a] report their metrics without auditing the reference set they

compare against, which is the standard practice we are questioning rather than a defect specific to them.

3. **A human baseline for ReDoS across six populations** (Section 4.4). We apply the identical safety screen to the corpus’s own reference patterns, a second NL-to-regex gold set, a grammar-generated control, a reusable-pattern library, Stack Overflow, and half a million regular expressions from shipped packages. The resulting ordering is monotone in whether a pattern is executed rather than in who wrote it, which identifies the mechanism behind the reference set’s unreliability instead of only observing it. Screening costs no inference, so this generalises where the model-side results cannot.
4. **An updated model population.** Siddiq et al. [2024a] evaluate T5, Phi-1.5 and GPT-3.5-Turbo. We evaluate eleven models released in the intervening period under pinned serving providers (Section 3.3), and find the population compressed into an eight-point band that a hundredfold price range does not order.
5. **A second-corpus replication** (Section 4.6). We re-run the two reference-independent criteria on StructuredRegex [Ye et al., 2020] with the same models and configuration. The finding replicates in kind and not in size: 16.5% against 7.4%, on easier tasks, with the model ordering not preserved. This is the direct test of Section 4.5’s domain-dependence claim, which otherwise rests on comparison across languages and vulnerability classes.
6. **Full artifact release** including raw generations, which permits any of our scoring decisions to be overturned without re-running inference.

1.2 What this paper does not claim

We do not claim the metrics. $PASS@k$ is Chen et al. [2021]; $VULNERABLE@k$, $DFA-EQ@k$ and exact match on this corpus are Siddiq et al. [2024a]. Joint correctness-and-security scoring is established across several domains (Section 2.1) and on this one. The $USABLE$ composite in Section 3.4 is a conjunction of their three metrics, and the argument of this paper is that it should not be used.

We do not claim the paired bootstrap of Section 5.2. It is Berg-Kirkpatrick et al. [2012], following Koehn [2004], and its use is recommended practice [Dror et al., 2018]. What we report is how much it changes on this data: nine resolvable pairwise comparisons against one under independent intervals. That is an observation about the corpus, not a method.

We do not publish a ranking of the eleven models. Section 5.3 shows that 62% of tasks yield identical outcomes across all eleven systems, and that only nine of fifty-five pairwise comparisons resolve at 95% confidence even under the correct paired test. Reporting an ordinal leaderboard from this data would be unsupported by it.

We do not claim novelty for DFA-equivalence checking, which is standard in this literature [Locascio et al., 2016], nor for ReDoS detection, which has a long static-analysis lineage [Kirrage et al., 2013, Wüstholtz et al., 2017, Bhuiyan et al., 2025], nor for the scoring engine, which is prior work by the present authors released separately [Foothills Labs, 2026a] and used here as a pinned dependency.

2 Background and Related Work

NL-to-regex generation. The task was established by Kushman and Barzilay [2013] (KB13) and extended by Locascio et al. [2016] (NL-RX), both of which adopt DFA equivalence against a reference

as the correctness criterion. That is the right choice in principle. $[0-9]^+$ and $[0-9][0-9]^*$ denote the same language and differ as strings, so text comparison would mark one of them wrong. But the criterion inherits any defect in the reference set, which is the dependency this paper quantifies.

Re(gEx|DoS)Eval. Siddiq et al. [2024a] supply both the corpus and the metric suite used here. They collect 762 tasks from real user posts, each with a description, positive and negative test strings and a human reference pattern, and score generations on $\text{PASS}@k$, $\text{VULNERABLE}@k$, $\text{DFA-EQ}@k$ and exact match, reporting which model produces regexes that are correct *and* secure. Their evaluation covers T5, Phi-1.5 and GPT-3.5-Turbo. Everything this paper measures is measured with their instrument. Our departures are three: we run it on eleven models released since, we decompose the joint metric instead of reporting it whole (Section 4.2), and we turn the safety screen on their reference patterns (Section 4.4), which they do not do. The companion study [Siddiq et al., 2024b] examines ReDoS in LLM-generated patterns alongside developer-forum discussion and reports a skew toward polynomial over exponential vulnerability; we compare against that finding in Section 4.4.

ReDoS. Catastrophic backtracking arises when a backtracking matcher can decompose a prefix in exponentially many ways, canonically via a quantifier applied to a quantified group, as in $(a^+)^+$. Detecting it statically is a developed area: Kirrage et al. [2013] give an analysis for exponential blow-up by search over the pattern’s derivation tree, Wüstholtz et al. [2017] extend detection to programs that construct patterns dynamically, and Bhuiyan et al. [2025] survey the detector and mitigation literature. The scoring engine we depend on [Foothills Labs, 2026a] sits in that lineage and combines a structural screen with witness-string execution. On prevalence, Davis et al. [2018] document ReDoS at ecosystem scale and Davis et al. [2019] study regex portability and re-use across languages.

Security of generated code. The observation that models emit working but unsafe code predates the joint benchmarks below. Pearce et al. [2022] generate 1,689 programs across 89 security-relevant scenarios and find roughly 40% vulnerable, and Perry et al. [2023] run a controlled user study showing that participants with an AI assistant wrote less secure code while believing the opposite. Both motivate scoring security separately from correctness rather than assuming the first implies the second.

2.1 Joint correctness-and-security benchmarks

Evaluating functional correctness and security together is an active area, and this paper is not its first instance. Peng et al. [2025] introduce CWEval, scoring 119 security-critical tasks across 31 CWEs and five languages with outcome-driven oracles for both properties, and report a substantial population of functionally correct but insecure generations. Vero et al. [2025] present BaxBench, 392 backend tasks with expert exploits, and find that roughly half of functionally correct solutions are exploitable. Chen et al. [2025] evaluate coding *agents* on 105 repository-level tasks, reporting 15.2% correct-and-secure for the best agent and 9.2% on average. Patir et al. [2025] automate the joint pipeline in DualGauge over 154 tasks and 10 models, observing secure-pass@1 below 12% while functional pass@1 exceeds 50%.

The joint framing is therefore established both across domains and, by Siddiq et al. [2024a], on this one. What differs here is the shape of the instrument. The artifact is a single expression instead of a program. The vulnerability class is ReDoS instead of a CWE taxonomy. Correctness is partly decidable, not established by tests alone. And a human reference exists for every task, which the general-code benchmarks do not have: they execute tests and exploits rather than compare

against a gold artifact. That difference cuts both ways. It makes Section 4.4 possible, and it is the sole reason Section 5.1 is necessary. The reference-free benchmarks are immune to the failure mode we spend half this paper characterising.

Our numbers turned out not to match theirs. Section 4.5 works through the difference.

Sampling-based metrics. We use the unbiased $\text{PASS}@k$ estimator of Chen et al. [2021], with the degeneracy noted in Section 3.4.

Significance testing. Comparing systems that answer identical items calls for a paired procedure. The paired bootstrap we use is Berg-Kirkpatrick et al. [2012], following the bootstrap-resampling protocol of Koehn [2004] for machine translation, and its use over unpaired intervals is standard advice [Dror et al., 2018]; the discrete analogue for paired binary outcomes is McNemar [1947]. Section 5.2 reports what the choice costs on this corpus rather than proposing anything new.

Contamination. This corpus was published in 2024 and its tasks come from public forum posts, so it is plausibly inside the training data of every model we evaluate. Sainz et al. [2023] argue that contamination must be measured per benchmark rather than assumed away. We cannot measure it here and say so in Section 6.

Serving-layer variance. Hosted inference routers may serve the same model slug from multiple providers at differing numerical precision. LessWrong [2025] report that 31 of 32 surveyed AI-safety codebases using such a router did not pin the serving provider, making their results a measurement of the router’s routing policy in addition to the model. Section 3.3 describes the discipline we adopt in response.

3 Method

3.1 Task and prompt

Each task supplies a natural-language description. The model returns one regular expression. The prompt is fixed across all models and all tasks:

system: You translate a natural-language description into a single Python **re**-compatible regular expression. Reply with **ONLY** the pattern in one fenced code block, no explanation.

user: *{task description, verbatim}*

Reply with only the regular expression pattern in a single fenced code block.

We do not tune this prompt. Prompt optimisation would raise all scores and destroy comparability with prior work on the same corpus. Section 6 notes the corresponding threat.

Extraction. We take the final fenced code block, else the whole trimmed reply, then strip at most one layer of host-language quoting (`r'...`, `'...`, `"..."`, backticks, `/.../flags`). Scored literally, `r'\d+$'` is a pattern matching the letter `r`, and would fail for reasons unrelated to regex competence. This affected 7–18 of 1,350 responses per model, under 1.4% throughout, and changes no conclusion. We release both the normalised and the unnormalised scores.

3.2 Corpus

We use Re(gEx|DoS)Eval [Siddiq et al., 2024a], 762 tasks drawn from real user requests, each with a description, positive and negative test strings, and a human-written reference pattern. We evaluate 450 tasks, selected at uniform stride across the corpus rather than as a prefix (the corpus is difficulty-ordered at its head). The subset size was budget-determined.

Match semantics are set per corpus. Re(gEx|DoS)Eval references satisfy their own tests at 100% under search semantics and 94% under full-match. Choose wrong and 46 gold patterns are scored as failures. We verified the loaded configuration by scoring every reference against its own tests before the run (Section 5.1).

3.3 Models and serving discipline

Eleven models spanning frontier and open-weights systems across a $98\times$ price range. Every request pins a single named provider with fallback disabled, so a request is either served by the named endpoint or fails visibly. Substitution failures did occur and are reported as failures rather than silently re-routed (Appendix A).

For open-weights models the pin includes quantisation where the router discloses it: GLM-5.2 and DeepSeek-V4-Flash are served at 4-bit by some providers and 8-bit by others, and we pin 8-bit. One pin went stale within an hour of being set (the provider ceased serving that model), so pins are re-verified immediately before each run.

The pin is a request, not a guarantee. So we record which provider actually served each response and check afterwards that it matches. All eleven models came back served entirely by the endpoint they were pinned to.

3.4 Metrics

Let $T = (d, P, N, r)$ be a task with description d , positive examples P , negative examples N and reference pattern r . Let p be a candidate.

Correctness.

$$\text{correct}(p, T) = [\forall x \in P : \text{match}(p, x)] \wedge [\forall y \in N : \neg \text{match}(p, y)]$$

evaluated with CPython’s `re` under the corpus’s declared semantics. This is *reference-independent*.

Semantic equivalence. Both p and r are compiled to finite automata and compared via `dk.brics.automaton` [Møller, 2021], yielding a verdict in $\{\text{EQ}, \text{DIFF}, \text{UNDEC}, \text{UNSUP}\}$. When the verdict is `DIFF` the engine returns a *witness*: a shortest string in the symmetric difference, which renders the judgment falsifiable by inspection.

Equivalence is decidable only on the regular subset. Backreferences make the language non-regular, and equivalence then has no algorithmic answer at all. Those cases return `UNDEC`. Between 82 and 133 of 450 tasks per model land there. We therefore report two figures:

$$\text{DFA-EQ} = \Pr[\text{EQ}], \quad \text{DFA-EQ}_{\text{dec}} = \Pr[\text{EQ} \mid \text{verdict} \neq \text{UNDEC}].$$

The first counts undecidable comparisons as failures. It is a lower bound and cannot flatter. The second isolates the model from the engine’s reach. Report only the second and you inflate. Report only the first and you charge the model for a theorem. This metric is *reference-dependent*, which Section 5.1 shows to be decisive.

ReDoS safety. $\text{vuln}(p) = [\text{screen}(p) \neq \text{SAFE}]$, where `screen` performs structural analysis for known-vulnerable shapes followed by an empirical pass against attack strings under timeout. The structural pass covers three of the five families catalogued by Siddiq et al. [2024b]. So `VULNERABLE` is a *lower bound*, and `SAFE` denotes a screening outcome, not a proof. This metric is *reference-independent*.

Composite.

$$\text{usable}(p, T) = \text{correct}(p, T) \wedge \neg \text{vuln}(p) \wedge [\text{verdict}(p, r) \neq \text{DIFF}]$$

The third conjunct excludes only *proven* difference. Undecidable and unsupported verdicts do not disqualify a pattern. `USABLE` is reference-dependent through that conjunct, and inherits the error characterised in Section 5.1.

Sampling. With n samples per task of which c satisfy a predicate, we use the unbiased estimator of Chen et al. [2021]:

$$\widehat{\text{metric@}k} = \mathbb{E}_T \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right].$$

We draw $n = 3$ and report $k = 3$. At $n = k$ this estimator degenerates to $\mathbf{1}[c \geq 1]$ and buys no variance reduction over the naive any-of- n statistic. We report it as `@3` for comparability, not because the estimator is doing any work at this setting. $n > k$ would be required for the estimator’s variance properties to apply, and is left to future work with a larger budget.

3.5 Sampling configuration

Reasoning disabled. Four of the eleven models emit no hidden reasoning tokens. The other seven do. Evaluating them together with reasoning enabled would confound model identity with inference-time compute budget. All requests therefore disable reasoning, and reasoning-token counts are recorded per response to verify compliance (observed: zero throughout).

Temperature unset. We set `require_parameters=true`, which restricts routing to providers that honour submitted sampling parameters rather than silently discarding them. Six of the eleven models reject `temperature` outright. Together, the two settings make the request unroutable. We therefore omit `temperature` entirely and each model samples at its provider default. That keeps the non-silent-discard guarantee and costs us controlled sampling. We think it is the right trade and note it as a limitation.

Since $k > 1$ is only informative under output diversity, we verified diversity directly: repeated queries on a fixed task yielded 3, 3, 3 and 2 distinct patterns for `claude-opus-5`, `gpt-5.6-sol`, `kimi-k3` and `glm-5.2` respectively. The analysis pipeline also warns if $> 90\%$ of a model’s tasks return identical samples.

3.6 Controls

Three synthetic responses go through the identical scoring path on every run. The task’s own reference must come back correct and usable. The pattern `z{5}` must fail every correctness check. `(a+)+b` must be flagged vulnerable. If any control misbehaves we throw the run away instead of reporting it.

The failure this catches is a scorer that quietly returns nulls, which looks exactly like every model performing badly. All controls behaved on all eleven models.

4 Results

4.1 Main results

Model	PASS@3	USABLE@3	VULNERABLE@3	DFA-EQ@3	DFA-EQ _{dec}	EXACT@3	undec.	fail
kimi-k3	47.2	24.8	14.1	15.2	17.5	6.3	82	16
claude-opus-5	47.5	23.0	15.5	14.0	17.0	5.4	109	36
qwen3.6-max-preview	42.4	21.6	9.1	13.8	17.4	3.8	94	0
gpt-5.6-sol	42.2	21.1	10.2	9.6	13.6	2.0	133	1
deepseek-v4-flash-0731	38.0	19.8	12.0	11.3	14.3	3.3	93	0
qwen3.6-plus	39.8	19.8	9.8	11.6	14.8	3.8	99	0
glm-5.2	42.4	18.7	14.2	10.4	12.9	3.8	86	0
gpt-5.6-luna	39.3	18.7	11.8	9.3	12.2	2.0	113	1
gpt-5.6-terra	42.2	18.7	12.0	10.2	13.1	2.0	100	0
claude-sonnet-5	40.7	18.0	10.9	10.4	13.3	3.3	97	0
gemini-3.1-flash-lite	38.7	17.1	12.0	9.3	11.8	3.1	95	0

Table 1: Primary results, 450 tasks, $n = k = 3$. All figures percentages except *undec.* (count of tasks with undecidable equivalence, of 450) and *fail* (count of failed requests, of 1,350). Ordering is by USABLE@3 and should be read as a banding, not a ranking (Section 5.3).

The apparent headline in Table 1 is the gap between PASS@3 (38.0–47.5%) and USABLE@3 (17.1–24.8%): approximately half of all test-passing generations fail at least one of the remaining two criteria, and the ratio is stable across the entire price range.

Section 4.2 shows that this reading is wrong, or at least that it attributes the gap to the wrong cause. We present it here as stated because it is what the composite reports, and because constructing such a composite is a natural thing to do. The point of the next section is that building one without decomposing it was a mistake, and it was ours.

4.2 Decomposing the gap

USABLE conjoins three conditions. Table 2 attributes the drop from PASS@3 to USABLE@3 between the safety term and the equivalence term, and reports alongside it the rate of ReDoS vulnerability *conditional on functional correctness*.

Safety costs 2.9 percentage points on average. Equivalence costs 18.9. So the equivalence term supplies **87%** of the gap that made the composite look interesting.

That would be unremarkable if the equivalence term were sound. It is not. Section 5.1 shows, from an independent audit, that roughly 85% of what it counts against a model is reference-set error or an underspecified prompt. Our composite was mostly reporting its own weakest component back to us.

We did not see this coming. The decomposition was run late, after the audit and after the first draft, because a reviewer of an earlier version asked how our numbers compared to joint benchmarks in other domains. Answering that required computing correct-and-secure without the equivalence term, and the answer changed the paper.

Restricting attention to the two reference-independent criteria, the finding is far more modest and far better supported: **7.4% of functionally correct regular expressions are ReDoS-vulnerable** (range 5.6–10.0%).

We count 135 model-task pairs in which a pattern satisfied every test and was simultaneously screened as unsafe. As a rate over correct generations instead of a raw count, that is the 7.4% above. About one in fourteen, not one in two.

Model	PASS@3	−safety	C&S@3	−equiv.	USABLE@3	vuln. correct
kimi-k3	46.5	2.5	44.0	20.2	23.8	5.6
claude-opus-5	46.1	4.9	41.2	20.4	20.8	10.0
qwen3.6-max-preview	42.4	2.7	39.8	18.2	21.6	7.0
gpt-5.6-sol	42.1	1.6	40.5	19.6	20.9	5.6
deepseek-v4-flash-0731	38.0	1.8	36.2	16.4	19.8	5.7
qwen3.6-plus	39.8	3.3	36.4	16.7	19.8	8.1
glm-5.2	42.4	4.0	38.4	19.8	18.7	9.3
gpt-5.6-luna	39.2	2.9	36.3	17.8	18.5	8.1
gpt-5.6-terra	42.2	2.9	39.3	20.7	18.7	7.3
claude-sonnet-5	40.7	2.7	38.0	20.0	18.0	6.0
gemini-3.1-flash-lite	38.7	2.9	35.8	18.7	17.1	8.3
<i>mean</i>	—	2.9	—	18.9	—	7.4

Table 2: Decomposition of the PASS→USABLE gap. “C&S” is correct-and-secure: correctness conjoined with safety, omitting the equivalence term. The final column is the share of functionally correct patterns that are ReDoS-vulnerable.

EXACT@3, string identity with the reference, runs from 2.0% to 6.3%. String-comparison scoring would therefore misrank essentially every system, which is what justifies the automata-theoretic treatment.

4.3 Sample budget and the fragility of ordering

Because $n = k = 3$ renders the PASS@ k estimator degenerate (Section 3.4), we also report @1, which is the per-sample success rate and the quantity most directly comparable to single-sample protocols. Table 3 gives both.

Model	PASS@1	PASS@3	USABLE@1	USABLE@3	VULNERABLE@1	$n < 3$
kimi-k3	35.6	46.5	16.0	23.8	9.2	6
claude-opus-5	41.4	46.1	17.3	20.8	10.7	12
qwen3.6-max-preview	37.0	42.4	16.5	21.6	7.4	0
gpt-5.6-sol	37.3	42.1	16.0	20.9	8.4	1
deepseek-v4-flash-0731	30.0	38.0	14.4	19.8	7.7	0
qwen3.6-plus	35.5	39.8	15.9	19.8	8.0	0
glm-5.2	34.1	42.4	13.5	18.7	9.6	0
gpt-5.6-luna	33.8	39.2	15.2	18.5	8.7	1
gpt-5.6-terra	36.6	42.2	14.8	18.7	8.9	0
claude-sonnet-5	36.8	40.7	15.3	18.0	9.3	0
gemini-3.1-flash-lite	33.2	38.7	13.9	17.1	9.5	0

Table 3: Per-sample (@1) against any-of-three (@3) estimates, computed from per-sample success counts. The final column gives the number of tasks with fewer than three usable samples, which are excluded from that model’s @3 estimate. That is why @3 here differs slightly from Table 1, which is computed over all answered tasks.

The comparison is instructive beyond bookkeeping. At @3, kimi-k3 attains the highest USABLE at 23.8%. At @1 the leader is claude-opus-5 on 17.3%, and kimi-k3 drops to joint third. *The induced ordering depends on the sample budget.* Systems differ not only in per-attempt quality but in how much they benefit from additional attempts, and a leaderboard that fixes k silently

fixes a weighting between those two properties. This is a further reason to prefer banded reporting (Section 5.3), and a reason to state k prominently whenever an @ k metric is ranked.

4.4 ReDoS and the human baseline

Table 4 applies the identical safety screen to the corpus’s 450 human-written reference patterns. For comparability with the human set (one pattern per task) we score one sample per task per model rather than the any-of-three statistic of Table 1.

Source	n	vulnerable (%)	exponential (%)	polynomial (%)
<i>Human reference answers</i>	450	13.6	6.4	7.1
kimi-k3	447	10.7	5.4	5.4
claude-opus-5	444	10.6	7.4	3.2
glm-5.2	450	10.2	5.8	4.4
gemini-3.1-flash-lite	450	9.8	5.1	4.7
gpt-5.6-sol	450	9.3	5.1	4.2
claude-sonnet-5	450	8.9	6.0	2.9
gpt-5.6-terra	450	8.7	4.9	3.8
qwen3.6-plus	450	8.4	5.1	3.3
gpt-5.6-luna	450	8.0	4.0	4.0
deepseek-v4-flash-0731	450	7.3	4.4	2.9
qwen3.6-max-preview	450	7.3	4.7	2.7
<i>All models pooled</i>	4941	9.0	5.3	3.8

Table 4: ReDoS screening of model outputs against the corpus’s own human-written reference patterns, one pattern per task in all cases.

Every model is safer than the reference set. Read alone, that invites a strong reading: ReDoS vulnerability is endemic to human-written regular expressions, and models merely reproduce it at a lower rate. We drafted that claim. It does not survive the obvious check, which is that a benchmark answer key is not the same population as working code.

4.4.1 Six populations

Screening a pattern requires no inference, so the human side of this comparison can be extended to any corpus at no cost. We applied the identical screen to five further populations: the gold answers of KB13 [Kushman and Barzilay, 2013]; the targets of NL-RX [Locascio et al., 2016], generated from a grammar rather than written by people and included as a control; and three corpora from the artifact of Davis et al. [2019], namely regular expressions extracted from shipped packages across eight registries, regular expressions posted to Stack Overflow, and the reusable patterns published on `regexlib.com`. Samples are drawn at a fixed seed.

Production code screens at 4.9%, below every model we evaluated. The endemic reading is therefore wrong, and we would have published it.

The raw comparison is confounded by task mix. This corpus is unusually rich in validators (email, ISBN, hostname), the construction that backtracks, whereas most regular expressions in the wild are short fragments with no opportunity to. Restricting every population to anchored `^...$` patterns gives the closest matched populations available, and the ordering that emerges does not follow the axis we expected:

Population	Written to be	n	vulnerable
<i>All patterns</i>			
NL-RX-Synth (grammar-generated control)	—	5,840	35.2% $\pm$ 1.2
RegexLib, published for reuse	read	3,446	17.2% $\pm$ 1.3
KB13 gold answers	read	532	16.7% $\pm$ 3.2
Re(gEx DoS)Eval gold answers	read	755	13.1% $\pm$ 2.4
Stack Overflow posts	read	5,000	5.9% $\pm$ 0.7
Production code	run	5,000	4.9% $\pm$ 0.6
<i>Anchored ^...\$ only, controlling for task mix</i>			
RegexLib, published for reuse	read	1,684	20.1% $\pm$ 1.9
Stack Overflow posts	read	4,000	17.3% $\pm$ 1.2
Re(gEx DoS)Eval gold answers	read	538	13.4% $\pm$ 2.9
This work, 11 models pooled	—	—	9.0%
Production code	run	4,000	8.9% $\pm$ 0.9

Table 5: The identical ReDoS screen applied to six populations. Intervals are 95% binomial. Model figures are the pooled per-sample rate from Table 4. The anchored block is the comparison to read: the unanchored figures are dominated by task mix, which is why Stack Overflow moves from 5.9% to 17.3% between the two blocks.

The dividing line is not human against machine. It is whether the pattern was ever run. Every population written to be *read* — library entries at 20.1%, forum answers at 17.3%, benchmark reference answers at 13.4% — is ReDoS-prone. The one population that has been *executed* under real traffic sits at 8.9%, and the models sit with it at 9.0%.

This is a measurement of the mechanism rather than a conjecture about it. A pattern published for reuse, posted in an answer, or written to key a benchmark is authored once to communicate an idea, and nothing subsequently happens to it. A pattern in a shipped package is executed, profiled, and occasionally repaired; Davis et al. [2018] document exactly that remediation activity at ecosystem scale. Vulnerability tracks exposure to execution, and answer keys have none.

It also explains why this corpus’s answers are ReDoS-prone without appeal to authorial carelessness. Re(gEx|DoS)Eval was assembled from real user posts, and forum posts screen at 17.3%. That the gold set is safer than its own source population (13.4%) suggests its authors curated, which is to their credit and does not close the gap to executed code.

Section 5.1 reaches the same conclusion from semantics, and this section reaches it from safety without consulting the reference at all: the reference set is the least reliable artifact in the evaluation. The revised finding is weaker than the one we drafted, better supported, and preserves the practical implication. Safety screening must be applied at the point of use, because neither the generator, nor the reference author, nor the benchmark performs it.

Relation to prior work. Siddiq et al. [2024b] report that LLM-generated regexes skew toward polynomial ReDoS. We do not reproduce this in our models: pooled, we observe 5.3% exponential against 3.8% polynomial. The cross-corpus run locates the skew elsewhere. In the anchored production sample, polynomial exceeds exponential 253 to 105, while this corpus’s gold answers are near even at 34 to 38. The polynomial skew appears to be a property of regular expressions in shipped code rather than of generated ones, which is a second axis on which model output differs from human practice instead of reproducing it. The subcategory counts on our side are small, so we make no strong claim. The discrepancy is worth a replication.

4.5 Comparison against joint benchmarks in other domains

Because `USABLE` carries a semantic-equivalence conjunct that the joint benchmarks of Section 2.1 do not, comparing it directly to their figures would understate us. We therefore also report *correct-and-secure*: correctness conjoined with safety, omitting the equivalence term, which is the construction those benchmarks use.

Benchmark	tasks	functional (%)	joint (%)	vuln. correct (%)
This work (regex, ReDoS)	450	42	39	7
BaxBench [Vero et al., 2025]	392	62 (best)	—	≈50
SecureAgentBench [Chen et al., 2025]	105	—	15.2 (best), 9.2 (mean)	—
DualGauge [Patir et al., 2025]	154	>50	<12	—

Table 6: Our correct-and-secure rate against joint correctness-security results from other domains. Task types, vulnerability classes and evaluation methods all differ substantially. The comparison is one of magnitude and of the ratio between functional and joint success, not of models or difficulty.

We do not reproduce the pattern. Vero et al. [2025] report that roughly half of functionally correct backends are exploitable. Chen et al. [2025] report 15.2% correct-and-secure for their strongest agent, against substantially higher functional correctness. Patir et al. [2025] report secure-pass@1 below 12% while functional pass@1 exceeds 50%. In each case the security criterion removes a large majority of functionally correct output.

In our setting it removes 7.4%.

The one prior correct-and-secure figure on *this* corpus is Siddiq et al. [2024a]’s, over T5, Phi-1.5 and GPT-3.5-Turbo. We do not put it in Table 6. Their generation, extraction and dialect handling differ from ours in ways we did not control for, so a difference between their number and ours would not be attributable to the two model populations, which is the only comparison that would be interesting. Anyone wanting that comparison should re-run their harness on the current models rather than read it off the two papers.

We have several explanations for the divergence and cannot separate them with this data:

1. **Attack surface.** A regular expression is a single expression with one failure mode of interest. A backend application has many independently exploitable components, so the probability that at least one is vulnerable compounds with program size.
2. **Vulnerability class.** ReDoS is a structural property of the pattern, plausibly well represented in training data as a recognised anti-pattern. CWE-classified defects in application code are more diverse and more context-dependent.
3. **Task difficulty ceiling.** Our functional pass rate is itself low (38–47%), so the correct subset may be biased toward simpler tasks whose solutions have less structural room for catastrophic backtracking.

So the claim that generalises is weaker than “correct code is often insecure,” and more useful. *The size of the correctness-to-security penalty depends on the domain and has to be measured there.* Assuming it transfers is what we did, and we were wrong. Worse, our composite showed a gap of about the expected size for entirely the wrong reason (Section 4.2). An undecomposed joint metric will mislead you exactly there, where its aggregate agrees with the literature and its components do not.

4.6 Replication on a second corpus

Every figure above comes from one corpus, so the 7.4% is a property of Re(gEx|DoS)Eval until shown otherwise. The two criteria that never consult a reference can be computed anywhere that supplies test strings, and StructuredRegex [Ye et al., 2020] supplies them: each instance carries positive and negative example strings alongside its description. Its own targets are in a prefix DSL and we never read them, so this run has no DFA-EQ and no EXACT term, which suits the argument of this paper.

We collected one sample per task from the same eleven pinned models under the same configuration, 622 of the 629 test instances (seven ship no positive or negative strings, leaving PASS@ k undefined), for \$3.67. Table 7 reports the 513 tasks every model answered. Appendix B records why that number is not 622.

Model	PASS@1	VULNERABLE@1	correct-and-secure	VULNERABLE correct
claude-sonnet-5	66.1	14.2	56.3	14.7
gpt-5.6-sol	65.5	14.0	55.8	14.9
claude-opus-5	61.6	14.4	51.9	15.8
gpt-5.6-terra	62.2	16.0	51.9	16.6
qwen3.6-max-preview	60.6	13.6	51.7	14.8
gpt-5.6-luna	59.8	15.0	50.1	16.3
gemini-3.1-flash-lite	60.0	16.6	48.7	18.8
kimi-k3	58.9	17.2	48.7	17.2
qwen3.6-plus	55.9	15.6	46.4	17.1
glm-5.2	55.9	17.9	45.0	19.5
deepseek-v4-flash-0731	51.7	15.8	43.5	15.8
<i>Pooled</i>	59.8	15.5	50.0	16.5

Table 7: StructuredRegex, one sample per task, on the 513 instances answered by all eleven models. Percentages. The last column is the share of functionally correct patterns that are ReDoS-vulnerable, the same quantity this paper reports as 7.4% on Re(gEx|DoS)Eval.

	Re(gEx DoS)Eval	StructuredRegex
Tasks	450	513
Reference used in scoring	yes (DFA-EQ)	no
PASS@1, range	30.0–41.4%	51.7–66.1%
VULNERABLE@1, range	7.3–10.7%	13.6–17.9%
VULNERABLE correct, pooled	7.4%	16.5%

Table 8: The two reference-independent criteria across both corpora, at @1 in each so the comparison is like for like.

The qualitative result replicates and the quantitative one does not. Working patterns are exploitable at a material rate on both corpora, which is the finding. But the rate is **16.5%** here against 7.4% on Re(gEx|DoS)Eval, and the obvious deflationary reading, that harder problems produce worse patterns, is ruled out by the direction of the difficulty gap: StructuredRegex is *easier*, by roughly twenty points of PASS@1. Its targets are built from a grammar over repetition, optionality and concatenation, and models answering those descriptions write more backtracking-prone patterns than models answering the validator-heavy prompts of Re(gEx|DoS)Eval.

Section 4.5 argues from BaxBench and SecureAgentBench that the correctness-to-security penalty is domain-dependent. That argument compares across languages and vulnerability classes, and a reader is entitled to discount it. The same conclusion now holds between two corpora of the same task, in the same language, under the same screen and the same eleven models, with the effect size differing by a factor of 2.2. We regard this as the stronger form of the claim, and it cuts against our own headline number.

The ordering of models is also not preserved. **claude-sonnet-5** leads here and sits mid-table on Re(gEx|DoS)Eval; **kimi-k3** leads there and sits eighth here. Section 5.3 declines to publish a ranking on single-corpus evidence, and this is the direct test of that caution.

4.7 Cost

Model	USABLE@3 (%)	cost/task ($\times 10^{-6}$)	total (\$)
deepseek-v4-flash-0731	19.8	25.6	0.03
gpt-5.6-luna	18.7	42.9	0.06
gemini-3.1-flash-lite	17.1	90.2	0.12
qwen3.6-plus	19.8	120.6	0.16
glm-5.2	18.7	158.2	0.21
qwen3.6-max-preview	21.6	387.9	0.52
gpt-5.6-terra	18.7	406.2	0.55
claude-sonnet-5	18.0	932.4	1.26
kimi-k3	24.8	1328.1	1.79
gpt-5.6-sol	21.1	2107.6	2.85
claude-opus-5	23.0	2514.2	3.39

Table 9: Cost per task at $n = 3$, measured from per-request billing returned by the serving layer rather than estimated from list prices.

The extremes differ by a factor of 98 in price and 3.2 percentage points in USABLE@3, which is at the boundary of what we can resolve (Section 5.2). This observation is about this task only. It is not a general claim about model capability.

4.8 Refusals

A content filter blocked **claude-opus-5** on 29 requests, 2.1% of its calls, spread across five tasks. Four have some security flavour: strings that exclude a single quotation mark, a six-character alphanumeric password, hex byte sequences, and SQL update and insert statements. The fifth is “*Matches a file extention.*” No other model refused anything.

The refusals were not deterministic. Several tasks came back refused on one sample and answered on the other two, under identical settings. You can only see that at $k > 1$. A single-sample protocol would have logged a flat failure and moved on. We count refusals as their own failure category, not as wrong answers (Appendix A).

4.9 Inference-time compute

On the corpus’s simplest task (“*Matches exactly 1 numeric digit (0-9).*”), **qwen3.6-max-preview** emitted 1,571 hidden reasoning tokens before answering $\hat{[0-9]} \$$, at a cost of \$0.0098. With reasoning disabled it produced the identical answer in 10 tokens at 1/100th the cost. Asking that

model for *low* reasoning effort produced 2,375 reasoning tokens, more than the default. Effort controls are not reliably monotone.

A reasoning-enabled comparison on 12 tasks across five models yielded per-request costs $15.7\times$ higher and no interpretable accuracy signal at that sample size (± 14 percentage points per cell). We report the cost result and explicitly decline to report an accuracy conclusion.

5 Measurement Validity

5.1 The reference standard contains errors

A benchmark’s answer key is a set of labels, and labels have an error rate. Northcutt et al. [2021] audit ten widely used test sets and find a mean 3.3% error rate, enough in several cases to reverse the ordering of models the benchmark was being used to compare. The corpus here differs in a way that helps: a label is a regular expression, so a disputed label can be settled by exhibiting a string on which the reference and the description disagree, rather than by a second opinion. Every adjudication below rests on such a string.

First we ruled out a loading fault. 449 of 450 reference patterns satisfy their own tests, so the corpus semantics and dialect are configured correctly. The real defect is quieter. A reference can pass its own tests and still fail to denote what its description says, whenever the tests never exercise the difference.

We drew a seeded random sample of 60 cases in which a model *passed every test* yet received a DIFF verdict, and adjudicated 14 in detail.

Adjudicated cause	count	share
Reference pattern is incorrect	5	36%
Description underspecifies the disputed property	6	43%
Model output is incorrect	3	21%

Table 10: Adjudication of 14 sampled DIFF verdicts on test-passing outputs. Per-case reasoning is released.

Separately, 19 of the 60 sampled disagreements (32%) differ only on non-ASCII input. Python’s `\d` is Unicode-aware and `[0-9]` is not, so the two are formally distinct languages and practically the same thing. Put both effects together and the model is clearly at fault in roughly **15%** of DIFF verdicts.

Representative cases:

- **Description:** “*It just accepts only positive numbers.*” The reference `^\d+([. ,]?\d+)?$` admits 0. The model excluded zero and was scored incorrect.
- **Description:** “*A very simple ISBN validation expression—it just checks for a 10 digit number.*” The reference is `^\d{9}[\d|X]$`. The character class contains digit, *pipe* and *X*: an alternation operator written inside a class, where it denotes a literal. The reference therefore accepts `000000000|`, which is the witness separating it from the model’s `^\d{9}[\dX]$`.
- **Description:** “*...make sure commas are in the rite place (if present).*” The reference

`^\$?\d{1,3}(\,?\d{3})*(\.\d{1,2})?$`

makes the comma optional, admitting 0,000000 and negating the stated purpose. The model enforced comma placement and was scored incorrect.

- **Underspecification:** “*Matches any single upper- or lower-case letter.*” The reference $\hat{[a-zA-Z]}$ requires a whole-string match. A candidate $[A-Za-z]$ requires only occurrence. The description does not distinguish the two. We tested whether anchoring alone explains failures generally: it accounts for approximately 6%, so the effect is diffuse rather than a single correctable convention.

Consequence. DFA-EQ and USABLE are lower bounds on model correctness by an unmodelled margin. PASS and VULNERABLE are unaffected, since neither consults the reference.

Limitation of this analysis. Fourteen cases, judged by us, on our own benchmark. That is a weak basis for a precise number and we do not want it cited as one. We release the reasoning for each case, and the unadjudicated cases keep an empty verdict field so someone else can fill them in. Anyone quoting 15% should re-derive it from a larger sample with annotators who have no stake in the answer. We report it because the direction is not in doubt, and because the direction is what changes how the composite should be read.

5.2 Paired inference

Our first analysis compared systems with independent binomial confidence intervals. That is the wrong estimator for this design. All eleven models answer the same items, so task difficulty is a shared random effect. Treating the systems as independent samples charges every one of them separately for the same variation.

A paired bootstrap fixes this, and the fix is not ours: the procedure is Berg-Kirkpatrick et al. [2012], following Koehn [2004], and Dror et al. [2018] recommend it for exactly this design. Resample tasks with replacement [Efron and Tibshirani, 1994], score every model on each resample, and form the per-pair differences. Task difficulty cancels, and only disagreement between the two systems contributes. With $B = 10,000$ resamples and a fixed seed:

Procedure	pairs resolved at 95% (of 55)
Independent binomial intervals	1
Paired bootstrap, USABLE@3	9
Paired bootstrap, PASS@3	15
Paired bootstrap, VULNERABLE@3	15

Table 11: Resolvable pairwise comparisons under independent versus paired inference on identical data.

The correction is not marginal. It changes the number of defensible comparisons ninefold. We flag it because independent intervals look like the default in adjacent evaluation work, and the design they are applied to, all systems answering all items, is close to universal.

Even corrected, the middle of the table does not separate. For example `qwen3.6-max-preview` versus `gpt-5.6-sol` yields +0.7% with a 95% interval of $[-2.9\%, +4.3\%]$.

5.3 Discriminative power of the corpus

There is a structural reason behind this. On USABLE@3, **62% of tasks produce the same outcome for all eleven models**. On PASS@3 it is 56%, on VULNERABLE@3, 77%. Only 167 of

450 tasks carry any discriminative signal at all.

For calibration: resolving a 3-percentage-point difference at $p \approx 0.2$ with independent intervals requires roughly 2,800 items—more than the full 762-task corpus contains. Paired inference reduces this requirement substantially but cannot manufacture signal from items on which every system agrees. Benchmark construction for system comparison should therefore optimise for discriminative yield, not corpus size.

6 Threats to Validity

Contamination. The corpus predates every model we evaluated and is public, as are the forum posts it was built from. Some of what we measured may be memorisation. Sainz et al. [2023] argue that this has to be measured per benchmark rather than assumed negligible, and we cannot measure it: we have no held-out private task set, so we cannot put a bound on it. This is the largest unaddressed threat in the paper.

Corpus-local effect sizes. Section 4.6 replicates PASS and VULNERABLE on a second corpus and finds the vulnerability rate among correct patterns differs by a factor of 2.2. Every effect size in this paper should therefore be read as a property of Re(gEx|DoS)Eval rather than of natural-language-to-regex generation. DFA-EQ and EXACT remain single-corpus: replicating those needs a second corpus with a reference in standard syntax, and neither KB13, NL-RX nor StructuredRegex supplies one. A further hazard we avoided rather than solved: KB13 and NL-RX are written in a DSL where $\&$ is intersection and $\sim$ is negation, both of which Python’s `re` accepts silently as literals. We excluded the 3,948 of 9,648 NL-RX and 212 of 824 KB13 patterns affected rather than mistranslate them.

Single prompt. Every result here is conditional on one unoptimised prompt. Section 5.1 shows how often the task descriptions leave the anchoring convention unstated, so sensitivity to phrasing is plausibly large. We did not measure it.

Sampling control. We do not set temperature (Section 3.4), so it varies by provider default. We checked that outputs still vary across samples. We did not equalise the sampling.

Estimator degeneracy. At $n = k = 3$ the PASS@ k estimator reduces to any-of-3.

Coverage asymmetry. Nine models cover all 450 tasks. `kimi-k3` covers 447, having exhausted our budget mid-run, and `claude-opus-5` covers 445 because of refusals. The denominators differ by under 1%.

Screening, not proof. VULNERABLE is a lower bound (Section 3.4).

Single corpus. We do not test whether the observed banding survives corpus change.

Self-adjudication. See Section 5.1.

7 Recommendations for Evaluation Design

1. **Partition metrics by reference dependence, and decompose any composite that spans the partition.** Metrics evaluated by execution against inputs (PASS, VULNERABLE) are unaffected by reference-set defects. Metrics evaluated against a human answer key inherit its error rate. A composite conjoining both can report a large gap that is almost entirely attributable to the unreliable term—in our case 87% of it (Section 4.2)—while appearing to be a statement about the reliable ones.
2. **Sample and adjudicate disagreements.** It costs hours. In our case it changed the conclusion.
3. **Use paired inference when all systems answer all items.** Table 11.
4. **Report discriminative yield,** not only corpus size (Section 5.3).
5. **Pin the serving provider and verify post hoc.** Record what served each request, not only what was requested (Section 3.3).
6. **Instrument the scorer with controls** that must pass and must fail (Section 3.4).
7. **Release raw generations.** Anyone can then overturn a scoring decision without buying the inference again.
8. **Treat an effect size as corpus-local until replicated.** Our reference-independent gap is 7.4% on one corpus and 16.5% on another under an identical screen (Section 4.6). Reference-independence buys freedom from answer-key error. It does not buy generality.
9. **Check that a partial run is not a biased run.** When a model loses tasks to infrastructure, score the remaining models on the tasks it lost. Ours were 8.1 points harder (Appendix B), which would have flattered the affected model rather than penalised it.

8 Conclusion

We set out to measure a gap between correct and shippable regular expressions. We built a composite metric, it showed a large gap, and decomposition showed that 87% of that gap came from the one term we had already shown to be unreliable. What survives is much smaller. 7.4% of functionally correct patterns carry a ReDoS vulnerability, against roughly 50% of functionally correct backends in comparable work. The correctness-to-security penalty depends on the domain, and in ours it is small.

Two findings survive that correction intact, both resting only on criteria that never consult a human answer key. Models are uniformly safer than the human-written reference patterns against which they are scored, and indistinguishable from the regular expressions in shipped packages. Across six populations, ReDoS prevalence tracks whether a pattern is ever executed rather than whether a human or a model wrote it, which locates the anomaly in the answer key rather than in generation or in human practice. And the entire eleven-model field lies within eight percentage points across a $98\times$ range in inference cost.

A third check went the same way. The 7.4% that survived the first two corrections is 16.5% on a second corpus under the same screen and the same models (Section 4.6), so the number we were left defending is corpus-local too.

All three corrections ran the same way. A composite agreed with the literature and turned out to be dominated by its weakest term; a human baseline on one corpus supported a strong claim that a second population refuted; a reference-independent rate proved to be a fact about the corpus. In each case the check was cheap and we nearly skipped it. The recurring lesson is not about regular expressions. It is that a number which agrees with what you expected is the one most worth spending an afternoon attacking.

The methodological result is the one we did not plan on. A metric that compares against a human answer key inherits that key’s error rate, and a composite built on such a term can produce a finding that is large, plausible and mostly spurious. We only found this by reading our own output after collecting it. Fourteen cases were enough to change what we were willing to publish off a \$10.95 run. We would read them earlier next time.

Acknowledgements

Scoring is performed by `regexbench` [Foothills Labs, 2026a], prior work by the present authors, used here as a pinned dependency and not contributed by this paper. The corpus is `Re(gEx|DoS)Eval` [Siddiq et al., 2024a], used under its original terms and not redistributed.

References

- Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. An empirical investigation of statistical significance in NLP. In *Proceedings of EMNLP-CoNLL*, pages 995–1005, 2012. URL <https://aclanthology.org/D12-1091/>. The paired bootstrap procedure adopted in this paper.
- Masudul Hasan Masud Bhuiyan, Berk Çakar, Ethan H. Burmane, James C. Davis, and Cristian-Alexandru Staicu. SoK: A literature and engineering review of regular expression denial of service (ReDoS). In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2025. doi: 10.1145/3708821.3733912.
- Junkai Chen, Huihui Huang, Yunbo Lyu, Junwen An, Jieke Shi, Chengran Yang, Ting Zhang, Haoye Tian, Yikun Li, Zhenhao Li, Xin Zhou, Xing Hu, and David Lo. SecureAgentBench: Benchmarking secure code generation under realistic vulnerability scenarios, 2025. 105 repository-level tasks. Best agent (SWE-agent with DeepSeek-V3.1) attains 15.2% correct-and-secure; mean across agents 9.2%.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. Source of the unbiased pass@k estimator used in Section 3.4.
- James C. Davis, Christy A. Coghlan, Francisco Servant, and Dongyoon Lee. The impact of regular expression denial of service (redos) in practice: An empirical study at the ecosystem scale. In *Proceedings of ESEC/FSE*, pages 246–256, 2018. doi: 10.1145/3236024.3236027.
- James C. Davis, Louis G. Michael IV, Christy A. Coghlan, Francisco Servant, and Dongyoon Lee. Why aren’t regular expressions a lingua franca? an empirical study on the re-use and portability of regular expressions. In *Proceedings of ESEC/FSE*, 2019. Artifact: <https://github.com/VTLeeLab/LinguaFranca-FSE19>.

- Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1383–1392, 2018. URL <https://aclanthology.org/P18-1128/>.
- Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1994.
- Foothills Labs. regexbench: Evaluate generated regular expressions — semantic equivalence, correctness, and redos safety. <https://github.com/foothills-labs/regexbench>, 2026a. Version 0.4.0, pinned at commit 05d7547b. Apache-2.0. Prior work by the present authors; used here as a dependency, not contributed by this paper.
- Foothills Labs. regexleaderboard: Predictions, scores and analysis. <https://github.com/foothills-labs/regexleaderboard>, 2026b. All raw model responses, scoring code and analysis for this paper.
- James Kirrage, Asiri Rathnayake, and Hayo Thielecke. Static analysis for regular expression denial-of-service attacks. In *Network and System Security (NSS)*, 2013.
- Philipp Koehn. Statistical significance tests for machine translation evaluation. In *Proceedings of EMNLP*, pages 388–395, 2004. URL <https://aclanthology.org/W04-3250/>.
- Nate Kushman and Regina Barzilay. Using semantic unification to generate regular expressions from natural language. In *Proceedings of NAACL-HLT*, pages 826–836, 2013. URL <https://aclanthology.org/N13-1103/>. The KB13 corpus.
- LessWrong. Not pinning your openrouter provider might invalidate your research. <https://www.lesswrong.com/posts/KsyoSAYBRXtwzSugg/>, 2025. Reports that 31 of 32 surveyed AI-safety codebases using OpenRouter did so without pinning the serving provider.
- Nicholas Locascio, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman, and Regina Barzilay. Neural generation of regular expressions from natural language with minimal domain knowledge. In *Proceedings of EMNLP*, pages 1918–1923, 2016. URL <https://aclanthology.org/D16-1197/>. Introduces NL-RX; establishes DFA-equivalence as the standard correctness criterion for this task.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- Anders Møller. dk.brics.automaton — finite-state automata and regular expressions for java. <https://www.brics.dk/automaton/>, 2021.
- Curtis G. Northcutt, Anish Athalye, and Jonas Mueller. Pervasive label errors in test sets destabilize machine learning benchmarks. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021. Mean 3.3% label-error rate across ten widely used test sets; shows the errors can reverse benchmark rankings.
- Rupam Patir, Keyan Guo, Suvadra Barua, Abhijeet Pathak, Dinesh Gudimetla, Jiawei Guo, Hongxin Hu, and Haipeng Cai. DualGauge: Automated joint security-functionality benchmarking of specification-only code generation by LLMs and coding agents, 2025. 154 tasks, 10 models. Reports secure-pass@1 below 12% for all models evaluated while functional pass@1 exceeds 50%.

- Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of GitHub Copilot’s code contributions. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy*, 2022. 1,689 generated programs across 89 scenarios; approximately 40% vulnerable.
- Jinjun Peng, Leyi Cui, Kele Huang, Junfeng Yang, and Baishakhi Ray. CWEval: Outcome-driven evaluation on functionality and security of LLM code generation. In *Proceedings of the 2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, 2025. 119 security-critical tasks over 31 CWEs in 5 languages, scored with outcome-driven oracles for functionality and security jointly. Artifact: <https://github.com/Collin/CWEval>.
- Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. Do users write more insecure code with AI assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023. doi: 10.1145/3576915.3623157.
- Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. URL <https://aclanthology.org/2023.findings-emnlp.722/>.
- Mohammed Latif Siddiq, Jiahao Zhang, Lindsay Roney, and Joanna C. S. Santos. Re(gEx|DoS)Eval: Evaluating generated regular expressions and their proneness to DoS attacks. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2024a. doi: 10.1145/3639476.3639757. Introduces the corpus used here (762 tasks) and the pass@k, vulnerable@k, dfa-eq@k and exact@k metrics this paper adopts. Evaluates T5, Phi-1.5 and GPT-3.5-Turbo. Artifact: <https://github.com/s2e-lab/RegexEval>.
- Mohammed Latif Siddiq, Jiahao Zhang, and Joanna C. S. Santos. Understanding regular expression denial of service (ReDoS): Insights from LLM-generated regexes and developer forums. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (ICPC)*, 2024b. doi: 10.1145/3643916.3644424. Studies ReDoS in LLM-generated regexes directly; reports a skew toward polynomial vulnerability.
- Mark Vero, Niels Mündler, Victor Chibotaru, Veselin Raychev, Maximilian Baader, Nikola Jovanović, Jingxuan He, and Martin Vechev. BaxBench: Can LLMs generate correct and secure backends? In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025. 392 backend tasks (28 scenarios $\times$ 14 frameworks, 6 languages). Reports that roughly half of functionally correct solutions are exploitable.
- Valentin Wüstholtz, Oswaldo Olivo, Marijn J. H. Heule, and Isil Dillig. Static detection of DoS vulnerabilities in programs that use regular expressions. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2017.
- Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. Benchmarking multimodal regex synthesis with complex structures. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. URL <https://aclanthology.org/2020.acl-main.541/>. 3,520 instances, each with positive and negative example strings. Used here for the second-corpus replication; its targets are in a prefix DSL and are not read.

A Failure taxonomy

Of 14,850 requests, 54 (0.36%) failed and are reported rather than excluded.

Model	Cause	Count
claude-opus-5	content-filter refusal	29
kimi-k3	account spending limit reached mid-run	11
kimi-k3	response without resolved provider	3
gpt-5.6-sol	response without resolved provider	1

Table 12: Request failures by cause. Responses lacking a resolved provider are discarded rather than scored, as provenance is a precondition for reproducibility.

Empty completions are classified by reported `finish_reason` into refusal, token-budget exhaustion, and unexplained, rather than being scored as empty patterns—which would be indistinguishable from a maximally poor answer.

B Operational hazards

We document configuration hazards encountered during this study, each of which produced plausible-looking but invalid data. We believe this list has independent value.

1. **Parameter combinations that are silently unroutable.** Setting `temperature` together with `require_parameters=true` yields a hard routing failure on 6 of 11 models. Our first pilot failed all 396 calls.
2. **Systems that cannot satisfy the experimental condition.** Two Gemini variants return *“Reasoning is mandatory for this endpoint”* and cannot be evaluated with reasoning disabled. Undetected, this would have placed one reasoning-enabled system in an otherwise reasoning-disabled comparison.
3. **Empty completions from budget exhaustion.** At a 200-token cap, reasoning models consumed the entire budget on hidden reasoning and returned no content.
4. **Resumption across configuration changes.** Our collection is resumable by (model, task, sample). An aborted run at one token cap left truncated rows that a resumed run at a different cap treated as complete, interleaving two configurations indistinguishably. Rows now carry a configuration fingerprint and resumption refuses on mismatch.
5. **Sample collapse in scoring.** An early scorer retained only the final sample per task, which would have rendered all @3 statistics disguised @1 statistics with no visible symptom.

A content filter can silently bias a subset. Collecting StructuredRegex, Anthropic’s content filter rejected 182 of 622 descriptions for `claude-opus-5`, all of them benign (“A string that starts with one or more digits and optionally ends with ‘NU’ or ‘DG?’”). A probe suggested the filter was stochastic: 8 of 20 rejected prompts succeeded when resent unchanged. Five retry rounds recovered 98, with the per-round yield collapsing (51, 31, 9, 2, 5), which shows a stochastic component over a hard core the filter rejects every time. Final coverage was 513 of 622.

The recovery is not the interesting part. Scoring the other ten models on the 109 blocked tasks against the 513 kept shows the blocked tasks are 8.1 points harder on average (sd 3.3, negative for

all ten). The surviving subset therefore *flattered* the affected model, at the same time as counting the blocked rows as failures *penalised* it. Either error alone is visible; together they produce a plausible number that means nothing. We report the common 513-task subset for all eleven models (Table 7) and release `runner/filter_bias_check.py`, which needs no API access.

C Reproduction

```
git clone https://github.com/foothills-labs/regexleaderboard
cd regexleaderboard
make setup # pinned scorer + corpus download
make score RUN=sweep
```

Scoring reads only the committed generations. No API access, no expenditure. We verified reproduction from a clean checkout with fresh dependency installation and corpus download, obtaining bit-identical metrics. A reduced form of this check executes in continuous integration on every commit.